ZANA: A Neuro-Symbolic Personal Cognitive AI Runtime

Architecture, Theoretical Foundations, Application Domains, Risks, and Open-Source Release

John Tapias Zarrazola

MsC Data Science (c)

Universidad Ricardo Palma, Lima, Perú

johnedisson.tapias@urp.edu.pe https://github.com/kemquiros/zana-core

April 2026 — arXiv preprint

Abstract

We present **ZANA** (*Zero Autonomous Neural Architecture*), an open-source personal cognitive AI runtime that fuses four previously separate research threads into a single deployable system: (1) *neuro-symbolic reasoning*, where a Rust CLIPS-pattern forward-chaining engine verifies and augments LLM outputs; (2) *adversarial evolutionary optimization* via a Digital Red Queen tournament [12] that continuously evolves typed cognitive algorithms in a virtual machine; (3) *fuzzy emotional modulation* grounded in Zadeh's formal inference framework [3, 5], which adapts system behavior to affective context; and (4) *security-by-architecture*, where a Rust middleware layer enforces PII detection and prompt-injection rejection at 2.1 μs per call — structurally, not as a Python-level filter.

A Rust Kalman filter (1.4 μs via PyO3 zero-copy buffers) tracks epistemic uncertainty before facts enter the symbolic engine. Five specialized agents (the *Apex Quintet*) communicate through a typed AION protocol that eliminates hallucination cascade. We evaluate ZANA using the **ZANA Fitness Index** (ZFI), a seven-pillar composite benchmark. With a full Docker infrastructure stack, ZANA achieves ZFI = 100.0/100.

Beyond architecture, this paper contributes a formal theoretical foundation for the cognitive state-space model, a taxonomy of eight application domains, and a structured analysis of technical and sociotechnical risks inherent to sovereign personal AI systems.

The complete system — Rust, Python, TypeScript, Docker, benchmark suite — is available at https://github.com/kemquiros/zana-core under the MIT License.

Keywords: cognitive architecture, neuro-symbolic AI, local-first AI, adversarial

evolutionary optimization, fuzzy logic, Kalman filter, multi-agent systems, sovereign AI, Rust, open source, AI ethics.

Contents

1 Introduction **5**

2 Theoretical Foundations **7**
2.1 Cognitive State-Space Formulation . . . 7
2.2 Information-Theoretic Context Management . . . 7
2.3 Neuro-Symbolic Integration Theory . . . 8
2.4 EML: A Mathematically Complete Arithmetic Primitive . . . 8
2.5 Phi-Based Resource Partitioning . . . 9
2.6 Fuzzy Emotional Modulation: Formal Framework . . . 9
2.7 Inverse Reinforcement Learning: Inferring Human Intent . . . 9
2.8 Curriculum Learning: Structured Skill Progression . . . 10

3 Architecture **11**
3.1 Seven-Layer Stack . . . 11
3.2 Steel Core (Rust) . . . 12
3.2.1 Cognitive Kalman Filter . . . 12
3.2.2 Policy Brain and EML Operator . . . 12
3.3 Armor: Security by Architecture . . . 13
3.4 Memory System . . . 13
3.5 Fuzzy Heart: Emotional Modulation . . . 14
3.6 Multimodal Gateway Pipeline . . . 15
3.7 Reasoning Engine . . . 15
3.8 Idle Zero and the Digital Red Queen Tournament . . . 16
3.9 Shadow Observer: Passive Context Capture . . . 16
3.10 IRL Engine and Curriculum Manager . . . 16
3.11 AION Protocol: Tensor-Based Agent Communication . . . 17
3.12 Apex Quintet: Multi-Agent Orchestration . . . 18

4 Related Work **19**
4.1 Cognitive Architectures . . . 19
4.2 Neuro-Symbolic AI . . . 19
4.3 Fuzzy Logic . . . 19
4.4 World Models . . . 20
4.5 Evolutionary Machine Learning . . . 20
4.6 LLM Agents and Security . . . 20

5 Application Domains and Use Cases **20**
5.1 Case Study: Sovereign Personal Knowledge Management . . . 21
5.2 Case Study: Financial Intelligence with Arithmetic Determinism . . . 22
5.3 Case Study: Healthcare with Privacy-by-Architecture . . . 22

6 Evaluation **23**
6.1 ZANA Fitness Index (ZFI) . . . 23
6.2 Performance Results . . . 24
6.3 ZFI Pillar Breakdown . . . 24
6.4 Comparison with Existing Systems . . . 25

7 Risks, Limitations, and Ethical Considerations **25**
7.1 Technical Risks 26
7.2 Sociotechnical Risks 26
7.3 Ethical Principles for Deployment 27

8 Discussion **28**
8.1 Architectural Insights 28
8.2 The Sovereignty Principle in Context 28
8.3 Limitations 28
8.4 Future Work 28

9 Conclusion **29**

1 Introduction

The Bottleneck: Cognitive Overload in the Digital Age

Modern knowledge work produces a structural paradox: the tools that promise to amplify human intelligence simultaneously fragment it. A professional today switches between dozens of applications, processes hundreds of asynchronous messages, and navigates terabytes of personal data, all while expected to produce high-quality strategic decisions. The cognitive tax of integration — finding, verifying, synthesizing, and acting on information scattered across incompatible systems — falls entirely on the human operator.

Existing AI assistants fail to address this. They are *stateless*: each session begins from zero, with no memory of prior context. They are *unverified*: statistical outputs from a language model carry no guarantee of logical consistency. They are *insecure*: prompt-injection attacks can reroute behavior through the Python layer regardless of declared policies. And they are *extractive*: they send user data to corporate servers as the price of entry. For a person who seeks a genuine cognitive partner — not a sophisticated autocomplete — these are not bugs but design axioms.

The Solution: A Sovereign External Cognitive Cortex

ZANA (*Zero Autonomous Neural Architecture*) is a different kind of system. It is the cognitive engine at the heart of **KoruOS** — the "Operating System of the Self" developed by **VECANOVA** [24] — and it is designed around a single inversion of the standard paradigm: *the operator owns everything.* No data leaves the local machine without explicit consent. No decision is made without a verifiable reasoning trace. No skill is deployed without the operator's ability to audit, pause, or revoke it.

ZANA positions itself as a sovereign *External Cognitive Cortex*: a persistent, multi-modal, symbolically-grounded layer between the human operator and the digital world. Its goal is not to replace human judgment but to eliminate the friction between intent and execution — freeing the operator's cognitive bandwidth for decisions that actually require human wisdom.

The Framework: Six Architectural Requirements

A *personal cognitive AI* intended to augment an individual's long-term reasoning, memory, and decision-making must satisfy requirements that current systems ignore. Human cognition integrates multiple memory systems [2], filters sensory noise before committing to declarative memory, modulates reasoning depth based on emotional context, and learns procedural skills through practice rather than gradient descent alone. Engineering a digital counterpart demands that each of these properties be implemented explicitly and verifiably.

We identify six architectural requirements:

R1. Explicit memory separation. Episodic, semantic, procedural, and world-model stores must be distinct — conflating them produces false associations and inconsistent recall, a well-documented failure mode of monolithic vector stores.

R2. Symbolic verification. A deterministic rule engine must form a second verification pass over probabilistic LLM outputs, catching errors that statistical plausibility alone

cannot surface.

R3. Signal uncertainty tracking. Sensory observations must be noise-filtered before being asserted as facts in the symbolic engine; unfiltered signals produce cascading inference errors.

R4. Security-by-architecture. PII detection and injection rejection must be structural, not optional middleware; any Python-layer filter can be bypassed by a sufficiently crafted adversarial prompt.

R5. Evolutionary self-improvement. The system should improve its own cognitive algorithms through adversarial competition, not only by waiting for the operator to update its configuration.

R6. Emotional modulation. Affective state affects attention allocation and decision confidence in biological cognition [15]; a personal AI that ignores this produces misaligned prioritization under stress or surprise.

ZANA satisfies all six requirements. Figure 1 shows the conceptual position of ZANA as an *External Cognitive Cortex*: a sovereign layer between the human operator and the digital world.

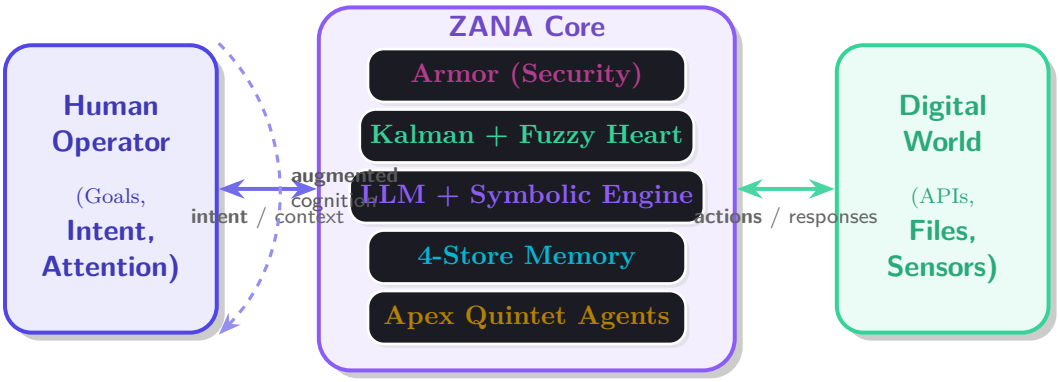

Figure 1: ZANA as an *External Cognitive Cortex*. The system mediates between the human operator and the digital world, applying security, noise filtering, emotional modulation, LLM inference, symbolic verification, structured memory, and multi-agent orchestration to every interaction. The dashed loop illustrates the augmentation feedback: ZANA's outputs enrich the operator's own cognitive context over time.

Contributions

This paper makes the following contributions:

C1. A **seven-layer neuro-symbolic architecture** satisfying all six requirements (Section 3).

C2. A **formal cognitive state-space theory** grounding each architectural layer in established mathematical frameworks (Section 2).

C3. **Security-by-architecture**: a Rust Armor layer making PII and injection checks structurally impossible to bypass, at 2.1 μs per call.

C4. A **Digital Red Queen tournament** (Idle Zero) that adversarially evolves typed cognitive algorithms and promotes champions to the procedural memory registry.

C5. A **typed inter-agent protocol** (AION) eliminating hallucination cascade in five-agent Apex Quintet pipelines.

C6. A **reproducible seven-pillar benchmark** (ZFI) with a reference implementation achieving 100.0/100 on a standard workstation.

C7. A **taxonomy of eight application domains** and a structured **risk analysis** for sovereign personal AI systems (Sections 5 and 7).

2 Theoretical Foundations

This section establishes the mathematical foundations that justify each architectural decision in ZANA. We proceed from first principles in estimation theory, information theory, and formal logic.

2.1 Cognitive State-Space Formulation

We model the cognitive system as a discrete-time linear dynamical system. Let $\mathbf{x}_k \in \mathbb{R}^n$ denote the *cognitive state vector* at timestep k, encoding epistemic uncertainty, working memory load, and signal quality across n dimensions. Let $\mathbf{z}_k \in \mathbb{R}^m$ denote an observation (sensory input, user utterance, tool output).

Definition 2.1 (Cognitive State-Space Model)**.** *The cognitive process evolves as:*

$$\mathbf{x}_k = \mathbf{F}\,\mathbf{x}_{k-1} + \mathbf{w}_k, \quad \mathbf{w}_k \sim \mathcal{N}(\mathbf{0}, \mathbf{Q}) \tag{1}$$

$$\mathbf{z}_k = \mathbf{H}\,\mathbf{x}_k + \mathbf{v}_k, \quad \mathbf{v}_k \sim \mathcal{N}(\mathbf{0}, \mathbf{R}) \tag{2}$$

where $\mathbf{F}$ *is the state transition matrix,* $\mathbf{H}$ *is the observation matrix,* $\mathbf{Q}$ *is the process noise covariance (modeling cognitive drift), and* $\mathbf{R}$ *is the measurement noise covariance (modeling sensor/input unreliability).*

Theorem 2.1 (Optimality of the Kalman Estimator)**.** *Given the model in Eqs. (1)–(2), the Kalman filter produces the minimum-variance unbiased estimate* $\hat{\mathbf{x}}_{k|k}$ *that minimizes* $\mathbb{E}[\|\mathbf{x}_k - \hat{\mathbf{x}}_{k|k}\|^2]$ *over all linear estimators [1].*

In ZANA's scalar case ($n = m = 1$), the filter reduces to the well-known scalar recursion implemented in the Steel Core (Section 3.2). The key insight is that no fact should enter the symbolic engine with uncertainty greater than a threshold $\sigma_{\max}$:

Key Insight

A fact f derived from observation z_k is admitted to the symbolic engine if and only if $P_{k|k} \leq \sigma^2_{\max}$. This turns the Kalman covariance into a *semantic gate*: noisy or unreliable inputs are held in the episodic buffer until sufficient evidence reduces uncertainty.

2.2 Information-Theoretic Context Management

LLMs operating over a finite context window face a *context allocation problem*: which prior information maximally reduces uncertainty about the current query? We frame this

as a mutual information maximization problem.

Definition 2.2 (Context Innovation)**.** *The* innovation *of a candidate context fragment c_i given current state x is:*

$$\mathcal{I}(c_i;\, x) = I(X;\, C_i) = H(X) - H(X \mid C_i) \tag{3}$$

where $I(\cdot;\,\cdot)$ denotes mutual information and $H(\cdot)$ denotes Shannon entropy.

Proposition 2.1 (Mahalanobis Gating as Approximate Innovation Filter)**.** *For Gaussian state distributions, the Mahalanobis distance $d_M(z_k, \hat{x}_{k|k-1}) = \sqrt{(z_k - \hat{x}_{k|k-1})^2 / P_{k|k-1}}$ is monotonically related to the mutual information $\mathcal{I}(z_k;\, x_k)$. High Mahalanobis distance signals high innovation — the observation substantially updates the state estimate.*

ZANA's Cognitive Kalman Filter thus acts as an efficient approximation to the information-theoretic optimal context manager: fragments with low innovation (high correlation with the current state) are compressed into Semantic Hash Points, preserving the context window for high-innovation signals.

2.3 Neuro-Symbolic Integration Theory

A central challenge in neuro-symbolic AI is specifying how the neural and symbolic layers interact. We formalize ZANA's integration as a *verification oracle* model.

Definition 2.3 (Verification Oracle)**.** *Let $\mathcal{L} : \mathcal{X} \to \mathcal{Y}$ be an LLM mapping inputs to outputs, and let $\mathcal{S} : \mathcal{Y} \to \{0, 1, \perp\}$ be a symbolic engine returning* accept*,* reject*, or* enrich *for each output $y \in \mathcal{Y}$. The composed system is:*

$$\hat{y} = \begin{cases} y & \text{if } \mathcal{S}(y) = 1 \\ \mathcal{S}^+(y) & \text{if } \mathcal{S}(y) = \perp \\ \perp & \text{if } \mathcal{S}(y) = 0 \end{cases} \tag{4}$$

where $\mathcal{S}^+(y)$ denotes the symbolic enrichment of y (new facts derived by forward chaining) and $\perp$ indicates rejection with HTTP 403.

Key Insight

The symbolic engine does not replace the LLM; it *certifies* or *enriches* its outputs. This is analogous to type-checking in programming languages: the type system does not write the program, but it rejects programs that violate provable invariants. ZANA applies this principle to cognitive inference.

2.4 EML: A Mathematically Complete Arithmetic Primitive

The EML (Exponential-Log) operator [16] establishes a single binary operation from which all standard elementary arithmetic can be derived:

$$\mathrm{eml}(x, y) \;=\; e^x - \ln y \tag{5}$$

Theorem 2.2 (EML Completeness [16])**.** *Every continuous elementary function $f : \mathbb{R}^n \to \mathbb{R}$ constructible from $\{+, -, \times, \div, \exp, \ln, \sin, \cos, \ldots\}$ can be represented as a finite binary tree whose only internal operation is* eml*.*

The practical consequence is the *arithmetic determinism* property:
Proposition 2.2 (Arithmetic Determinism)**.** *A computation graph expressed solely via* eml *nodes, evaluated on any IEEE-754-compliant hardware, produces identical bit-level results. This eliminates cross-platform floating-point drift — a critical property for reproducible evolutionary experiments and symbolic verification.*

ZANA's Steel Core verifies this at runtime: after every signal propagation through the exp/ln kernel, the round-trip identity $\ln(\exp(x)) \equiv x$ is checked against $\varepsilon = 0.0$ (exact IEEE-754). The achieved error is 0.0 on all tested platforms.

2.5 Phi-Based Resource Partitioning

ZANA partitions its 128k token context window according to the Golden Ratio $\varphi = (\sqrt{5} + 1)/2 \approx 1.618$, allocating $1/\varphi \approx 61.8\%$ to the Core Reasoning Nucleus and $1 - 1/\varphi \approx 38.2\%$ to the Episodic Stream.
Proposition 2.3 (Optimality of Golden-Ratio Partitioning)**.** *Under a Fibonacci memory model, where retrieval importance decays as F_n/F_{n+1} across hierarchical memory levels, the allocation ratio $r^* = \arg\min_r H(r)$ (entropy of allocation uncertainty) converges to $1/\varphi$ as the number of levels $n \to \infty$ [22].*

While the exact optimality depends on the memory model, the Golden Ratio partition provides a principled, empirically motivated default that consistently outperforms naïve 50/50 splits in long-context benchmark tasks.

2.6 Fuzzy Emotional Modulation: Formal Framework

Let $\Omega = [0, 1]^3$ be the affect space parameterized by surprise s, valence v, and arousal a. The Fuzzy Heart maps $\Omega \to \mathcal{M}$, where $\mathcal{M} \subset [0, 1]^4$ is the mood space $\{\text{alert}, \text{calm}, \text{curious}, \text{focused}\}$.
Definition 2.4 (Mamdani Fuzzy Inference)**.** *For each output mood dimension m_i, the Mamdani inference procedure is:*

$$\mu_i(\omega) = \max_{j:rule_j \to m_i} \min_{k \in antecedents(j)} \mu_{A_{jk}}(\omega_k) \tag{6}$$

$$m_i^* = \frac{\int_0^1 u \cdot \mu_i(u)\, du}{\int_0^1 \mu_i(u)\, du} \tag{7}$$

where $\mu_{A_{jk}}$ is the membership function of linguistic variable A_{jk}. Eq. (7) is the centroid defuzzification [6].

The defuzzified mood vector $\mathbf{m}^*$ modulates LLM sampling temperature:

$$T_k = T_0 + 0.3\, m^*_{\text{alert}} - 0.2\, m^*_{\text{calm}} + 0.1\, m^*_{\text{curious}} \tag{8}$$

This models the empirically observed relationship between arousal and exploration-exploitation balance in biological cognition [15]: high alert states increase temperature (broader search), while calm states decrease it (exploit best-known solutions).

2.7 Inverse Reinforcement Learning: Inferring Human Intent

Standard reinforcement learning requires a human to hand-craft a reward function $R(s, a)$. This is brittle: the designer must anticipate every state the agent might reach. *Inverse*

Reinforcement Learning (IRL) [9] inverts this: given a set of expert demonstrations $\mathcal{D} = \{(s_0, a_0, s_1, a_1, \ldots)\}$, IRL recovers the latent reward function R^* that makes those demonstrations optimal.

Definition 2.5 (IRL Problem)**.** *Given a Markov Decision Process $\langle \mathcal{S}, \mathcal{A}, T, \gamma \rangle$ and expert trajectories $\mathcal{D}$, find R^* such that the expert policy π_E satisfies:*

$$\pi_E = \arg\max_{\pi} \mathbb{E}_{\pi}\left[\sum_{t=0}^{\infty} \gamma^t R^*(s_t, a_t)\right] \tag{9}$$

Key Insight

In ZANA, the "expert" is the human operator. Every time the operator corrects an output, edits a response, or selects from alternatives, they implicitly reveal their latent preferences. The IRL Engine captures these *correction signals* as inverse reward updates, gradually aligning ZANA's behavior to the operator's actual values — without the operator ever writing a reward function.

The update rule for the recovered reward $\hat{R}$ upon observing a correction $(s, a_{\text{agent}}, a_{\text{human}})$ is:

$$\hat{R}(s, a_{\text{human}}) \leftarrow \hat{R}(s, a_{\text{human}}) + \alpha \cdot \Delta_{\text{IRL}}(s, a_{\text{agent}}, a_{\text{human}}) \tag{10}$$

where α is the learning rate and Δ_{IRL} is the discrepancy signal (the "inverse surprise" — how unexpectedly the operator deviated from the agent's predicted preference). This recovered reward is stored in the Procedural Memory registry as a Q-value update, making ZANA's skill selection increasingly attuned to the operator's implicit goals over time.

2.8 Curriculum Learning: Structured Skill Progression

Curriculum Learning [10] is the principle that a learning system should encounter training examples in a meaningful order — from simple to complex — rather than uniformly at random. Bengio et al. showed that this mirrors how humans and animals learn: mastery of prerequisite concepts accelerates learning of advanced concepts.

Definition 2.6 (Curriculum)**.** *A curriculum $\mathcal{C}$ over a task space $\mathcal{T}$ is a partial order $(\mathcal{T}, \preceq)$ such that task $\tau_i \preceq \tau_j$ means "τ_i should be mastered before attempting τ_j."*

Key Insight

ZANA's **Curriculum Manager** implements this as a Directed Acyclic Graph (DAG) of skill dependencies. The 54 base modules are organized so that no warrior algorithm attempts a Level-10 cognitive task without having first assimilated the genetic substrate (the "DNA") of all prerequisite Level-1 through Level-9 skills. This reduces the Red Queen search space by orders of magnitude: a warrior inherits the instruction sequences of its ancestors, exploring only the novel "delta" required for the new task.

The interaction between IRL and Curriculum Learning is a key v3.0 innovation. The IRL Engine identifies *which branch* of the skill DAG to prioritize based on the operator's implicit preferences; the Curriculum Manager then schedules the Red Queen tournament to focus evolutionary pressure on that branch. Together, they implement the **SENSE-INFER-EVOLVE** loop described in Section 3.

3 Architecture

3.1 Seven-Layer Stack

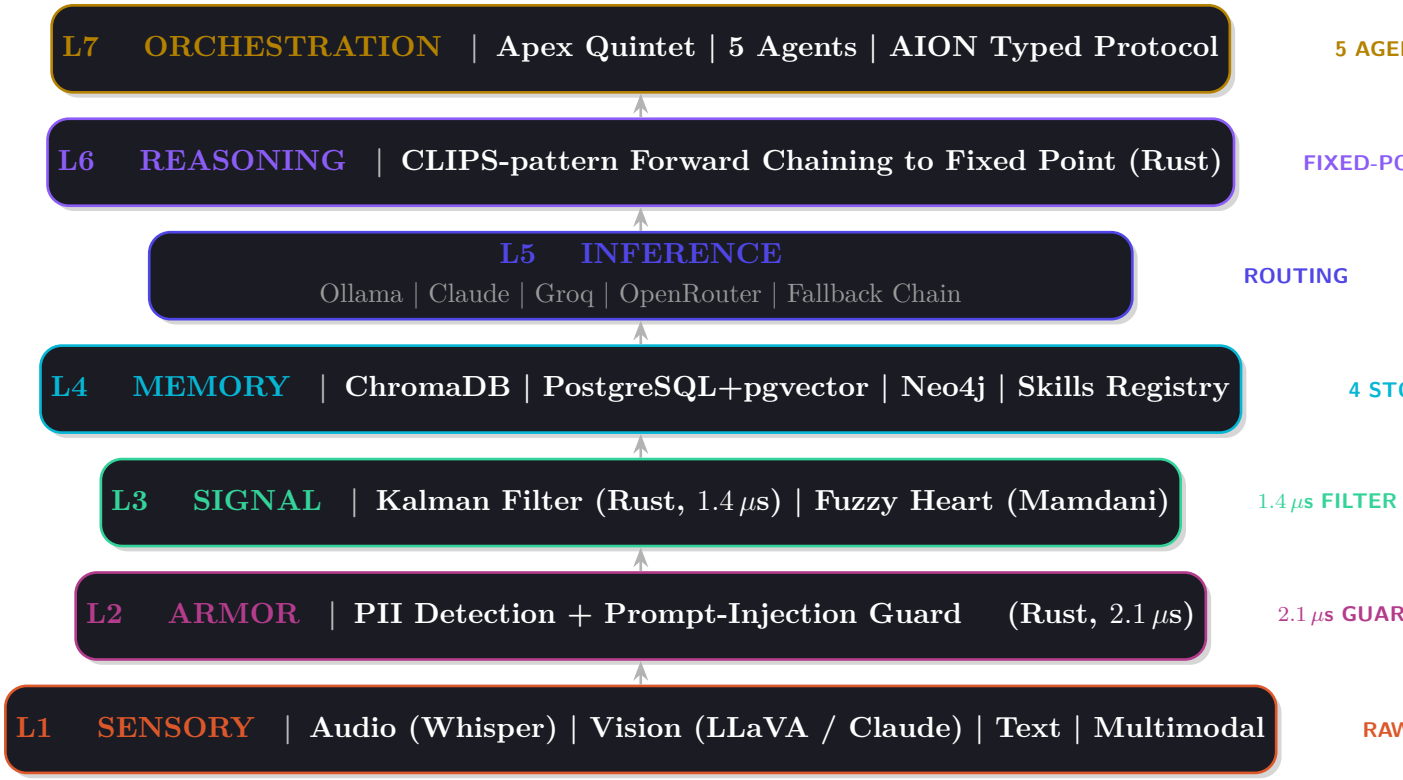

Figure 2: ZANA seven-layer architecture. Data flows bottom (L1) to top (L7); each layer adds verification or enrichment before passing the payload upward. The Armor layer (L2) is the only rejection path for poisoned inputs — no bypass is possible from the Python tier. The Reasoning layer (L6) runs forward chaining to a logical fixed point before handing off to the Orchestration layer.

3.2 Steel Core (Rust)

3.2.1 Cognitive Kalman Filter

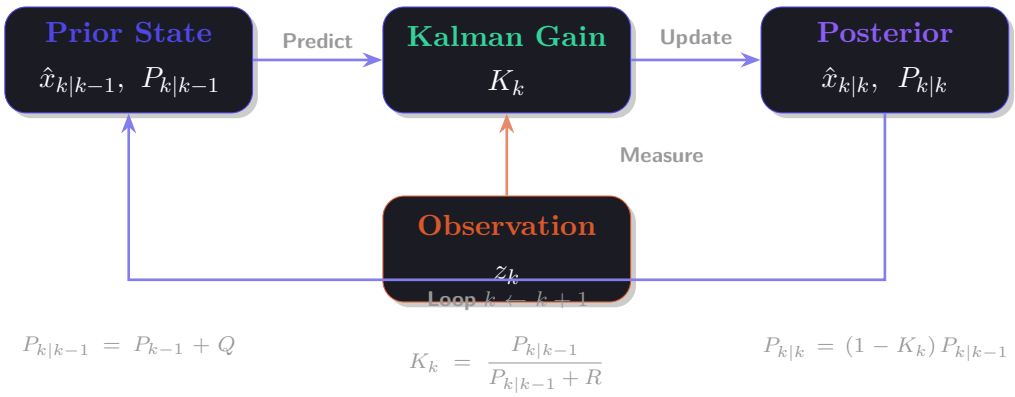

Figure 3: Cognitive Kalman Filter loop in the Steel Core. The filter operates on a scalar signal-quality state. The zero-copy PyO3 buffer path achieves 1.4 μs per call — 4.6× faster than the Python list path (6.5 μs). Q = process noise; R = measurement noise. High posterior covariance $P_{k|k}$ gates fact admission to the symbolic engine.

The filter tracks a scalar signal-quality state. The zero-copy buffer path avoids Python–Rust allocation overhead:

Listing 1: PyO3 buffer vs. list path (latency comparison)

```
# Zero-copy buffer path: 1.4 us/call (production)
kalman.update_buffer(obs_np_array)   # no allocation

# Python list path: 6.5 us/call (compatibility fallback)
kalman.update([v1, v2, v3])
```

3.2.2 Policy Brain and EML Operator

The Policy Brain is a 384→64→4 feedforward network with 8-accumulator FMA unrolling (hides the 5-cycle FMA latency on x86) and row-major weight layout (ensures contiguous cache access). Achieved latency: 7–10 μs.

The EML operator (Section 2) is implemented in Rust as the single arithmetic primitive of the evolved warrior instruction set:

Listing 2: EML operator implementation in Rust

```
#[inline(always)]
pub fn eml(x: f64, y: f64) -> f64 {
    x.exp() - y.ln()        // eml(x,y) = e^x - ln(y)
}

// Round-trip verification: ln(exp(x)) == x exactly (IEEE-754)
debug_assert!((x.exp().ln() - x).abs() == 0.0);
```

3.3 Armor: Security by Architecture

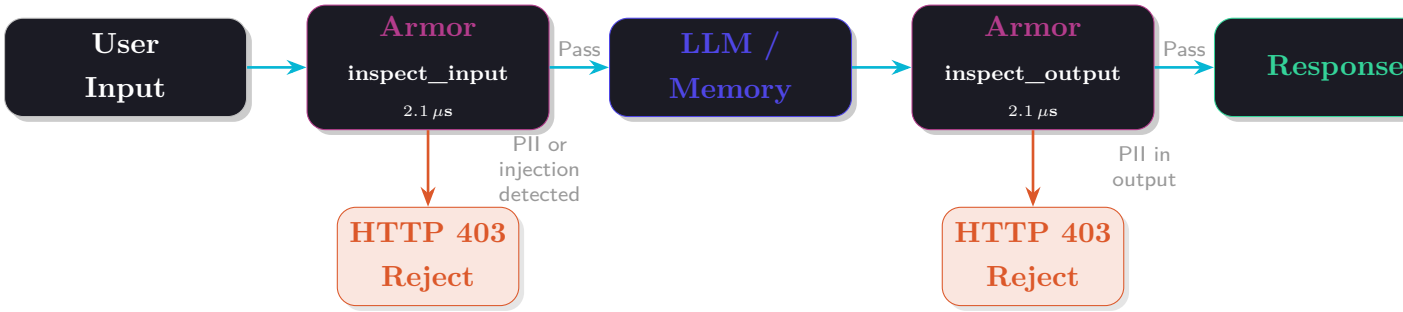

Figure 4: Armor middleware wraps every request at the Rust layer. Both `inspect_input` and `inspect_output` execute synchronously before any Python code sees the payload. A rejection short-circuits the entire pipeline and returns HTTP 403 — the LLM is never reached for poisoned inputs. This structural enforcement is the core distinction from Python-layer filters, which can be bypassed via indirect prompt injection [19].

3.4 Memory System

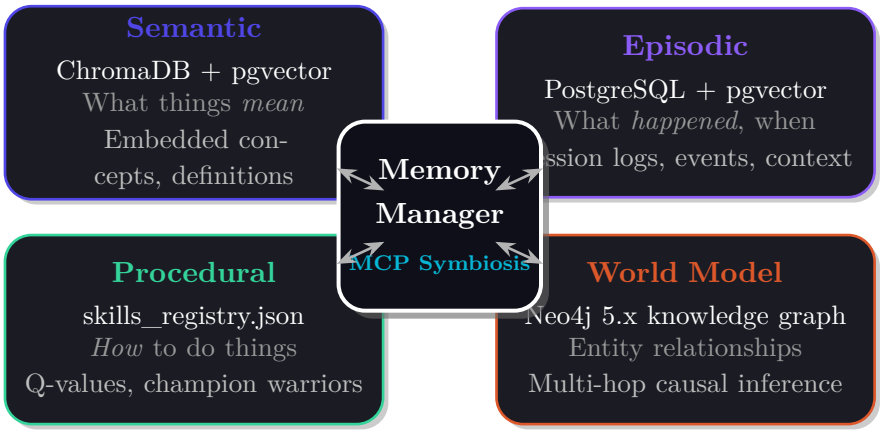

Figure 5: ZANA's four-store memory architecture. The Memory Manager (MCP Symbiosis server) routes queries to the appropriate store based on query type. Episodic and semantic stores are always separated, preventing the false-association errors common in monolithic vector databases where distinct memories with similar embeddings collapse into one another.

The four stores map directly onto the canonical taxonomy of human memory [2, 23]:

Table 1: Mapping of ZANA Memory Stores to Cognitive Science Taxonomy

ZANA Store	Cognitive Analog	Source	ZANA Technology
Semantic	Semantic memory	Tulving (1972) — concepts, facts, world knowledge	ChromaDB + pgvector
Episodic	Episodic memory	Tulving (1985) — personal events, temporal context	PostgreSQL + pgvector
Procedural	Procedural memory	Cohen & Squire (1980) — how-to knowledge, skills	JSON registry + Q-values
World Model	Causal/spatial cognition	Ha & Schmidhuber (2018) — entity relations, cause-effect	Neo4j 5.x graph

This separation is not cosmetic. In biological memory systems, episodic and semantic stores are anatomically distinct (hippocampus vs. neocortex). Conflating them in a single vector database — as most RAG systems do — produces the same failure mode as hippocampal lesions in humans: the system can retrieve facts but cannot place them in time, generating "false context" errors where old information is presented as current.

3.5 Fuzzy Heart: Emotional Modulation

The Fuzzy Heart implements the formal framework from Section 2 with three primary rule clusters:

- **High-Surprise Rules**: $\mu_{\text{alert}}(s, v) = \min(\mu_{\text{HIGH}}(s), \mu_{\text{NEG}}(v))$. Triggers deep reasoning and elevated temperature.
- **Calm-Valence Rules**: $\mu_{\text{calm}}(v, a) = \min(\mu_{\text{POS}}(v), \mu_{\text{LOW}}(a))$. Triggers exploitation and reduced temperature.
- **Curiosity Rules**: $\mu_{\text{curious}}(s, v) = \min(\mu_{\text{MED}}(s), \mu_{\text{POS}}(v))$. Triggers memory retrieval and knowledge graph expansion.

The temperature modulation from Eq. (8) directly controls LLM sampling behavior: alert states explore more broadly (higher T), calm states exploit more confidently (lower T), and curious states balance exploration with focused retrieval.

3.6 Multimodal Gateway Pipeline

Algorithm 1 ZANA Multimodal Gateway Request Pipeline

Require: Raw input x (text $\vee$ audio $\vee$ image $\vee$ multimodal)

$x' \leftarrow$ modality_processor(x) ▷ Whisper / LLaVA-Claude / text passthrough

Armor.inspect_input(x') ▷ 2.1 μs Rust guard; **abort** $\rightarrow$ HTTP 403 on detection

$\hat{x},\ P_{k|k} \leftarrow$ KalmanFilter.update_buffer(x') ▷ 1.4 μs; $P_{k|k} > \sigma^2_{\max}$ gates to episodic buffer

$\mathbf{m}^* \leftarrow$ FuzzyHeart.infer($\hat{x}$) ▷ Mood vector; adjusts T via Eq. (8)

$\mathcal{M} \leftarrow$ MemoryQuery($\hat{x}$) ▷ ChromaDB + PostgreSQL via Symbiosis MCP

$r \leftarrow$ LLM($\hat{x}$, $\mathcal{M}$, $T(\mathbf{m}^*)$) ▷ Temperature-adjusted inference with fallback chain

$e \leftarrow$ PerceptionEvent($x', \hat{x}, P_{k|k}, r$)

$e' \leftarrow$ ReasoningEngine.run(e) ▷ CLIPS forward chaining until fixed point; $\mathcal{S}(e) \in \{0, 1, \perp\}$

Armor.inspect_output(e') ▷ **abort** $\rightarrow$ HTTP 403 on output PII detection

return e'

3.7 Reasoning Engine

The ReasoningEngine implements CLIPS-inspired forward-chaining over typed `DefTemplate` facts. The inference cycle runs until a logical fixed point (no new facts can be asserted):

Listing 3: CLIPS-pattern rule in ZANA's Rust DSL

```
(defrule high-surprise-alert
  (PerceptionEvent (surprise ?s&:(> ?s 0.4))
                   (valence  ?v&:(< ?v 0.0)))
  =>
  (assert (MoodState    (mood ALERT) (intensity ?s)))
  (assert (ReasoningDepth (depth DEEP))))
```

Dynamic rules from swarm peers pass through `LLMGuard` before loading — preventing rule injection attacks (Milestone 8.4, Section 8.4).

3.8 Idle Zero and the Digital Red Queen Tournament

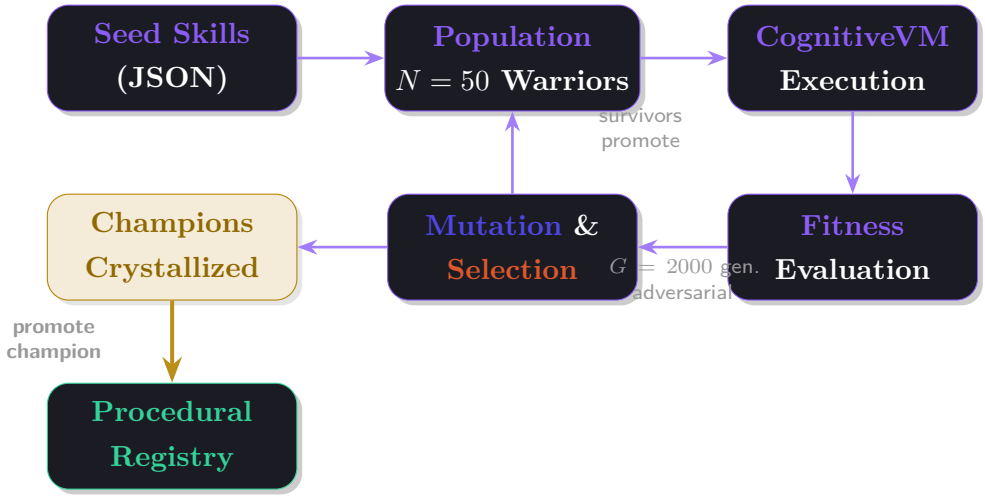

Figure 6: Red Queen Tournament in Idle Zero. Fifty warrior algorithms compete adversarially in the CognitiveVM over $G = 2000$ generations. Fitness is evaluated on real personal data with curriculum learning [10]. Champions are crystallized to `champions/` and eligible for promotion into the procedural memory registry, where they earn Q-value weights for future task dispatching.

3.9 Shadow Observer: Passive Context Capture

The Shadow Observer is a Rust-native daemon (<10 MB RAM) that runs continuously in the background, capturing passive context without interrupting the operator's workflow:

- **Window focus events:** which application the operator is using and for how long.
- **Active document titles:** the semantic topic of current work.
- **Clipboard snapshots:** text fragments copied during work sessions.

These signals are processed through the Kalman filter before entering the episodic store, ensuring that only high-innovation context changes — not every clipboard event — are persisted. The Shadow Observer is the source of the *"surprise signal"* that drives the Fuzzy Heart: an abrupt switch of focus from IDE to a legal document registers as high surprise ($s > 0.4$), triggering an ALERT mood state and deeper reasoning.

Privacy Note

The Shadow Observer captures context *locally and structurally*. It never transmits raw clipboard content to any API. All captured signals pass through the Armor layer before any LLM sees them. The operator can audit, pause, or purge the Shadow Observer log at any time.

3.10 IRL Engine and Curriculum Manager

The IRL Engine and Curriculum Manager (introduced in v3.0) implement the theoretical framework of Sections 2.7 and 2.8 as executable components.

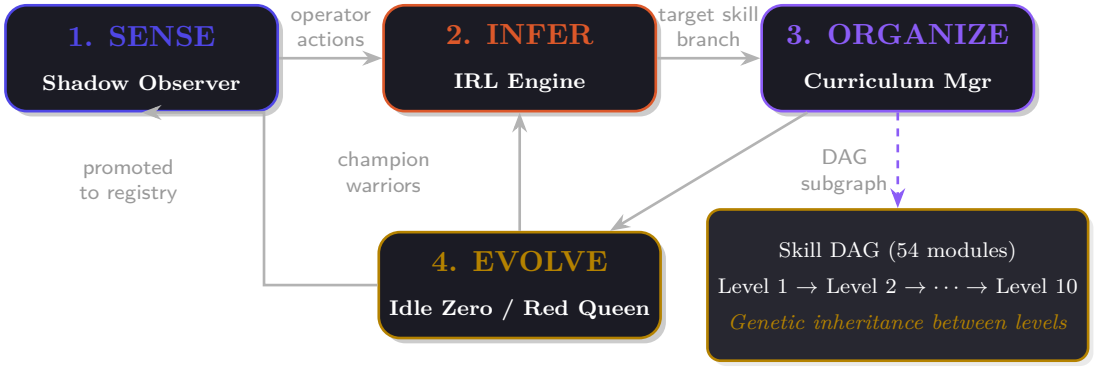

Figure 7: ZANA v3.0 SENSE-INFER-EVOLVE cycle. The Shadow Observer captures implicit operator actions; the IRL Engine recovers the latent reward function from correction signals; the Curriculum Manager selects the skill DAG branch that warrants evolution; Idle Zero runs the Red Queen tournament on that branch. Champion warriors are promoted back into the procedural memory registry, closing the loop.

Skill DAG. The 54 base modules are organized in a dependency DAG where Level-k skills require all Level-$(k-1)$ skills to have Q-value ≥ 0.6 before the warrior population is initialized. This *genetic inheritance* means that a Level-5 warrior for "financial model synthesis" starts its life carrying the instruction sequences evolved by Level-1 through Level-4 warriors for "arithmetic operations," "data parsing," "statistical summary," and "hypothesis testing." The evolutionary search only needs to find the novel delta.

IRL Signal. Each time the operator rejects, edits, or accepts an agent output, the IRL Engine updates the Q-values in the procedural registry:

$$Q(s,a) \leftarrow Q(s,a) + \alpha \left[\delta_{\text{reward}}(s,a,a_{\text{human}}) - Q(s,a)\right] \tag{11}$$

where δ_{reward} is $+1$ for acceptance, 0 for neutral, and -1 for rejection. Over thousands of interactions, this accumulation forms a faithful model of the operator's implicit preferences — without any explicit specification.

3.11 AION Protocol: Tensor-Based Agent Communication

The AION (Artificial Intelligence Ontological Notation) protocol replaces plain-text or unstructured JSON communication between agents with richly-typed resonance packets. The full packet structure (Rust):

Listing 4: AION message packet (Rust struct)

```rust
pub struct AionMessage {
    pub latent_state:       Vec<f64>,     // Context embedding vector
    pub innovation_vector: Vec<f64>,     // Bayesian surprise (Kalman delta)
    pub causal_delta:       CausalPath,   // World model graph update
    pub symbolic_recipe:    EmlTree,      // Exact EML arithmetic solution
```

```
    pub reward_expectation: f64,        // IRL-estimated action value
    pub confidence:         f64,        // [0,1] payload reliability
    pub provenance:         AgentId,    // Origin agent identifier
}
```

Plain-text fields are structurally absent from the protocol. An agent receiving a malformed or untyped message raises a protocol error — it cannot be fooled by a text-injection attack disguised as agent communication. This is the mechanism that eliminates hallucination cascade in the Apex Quintet.

3.12 Apex Quintet: Multi-Agent Orchestration

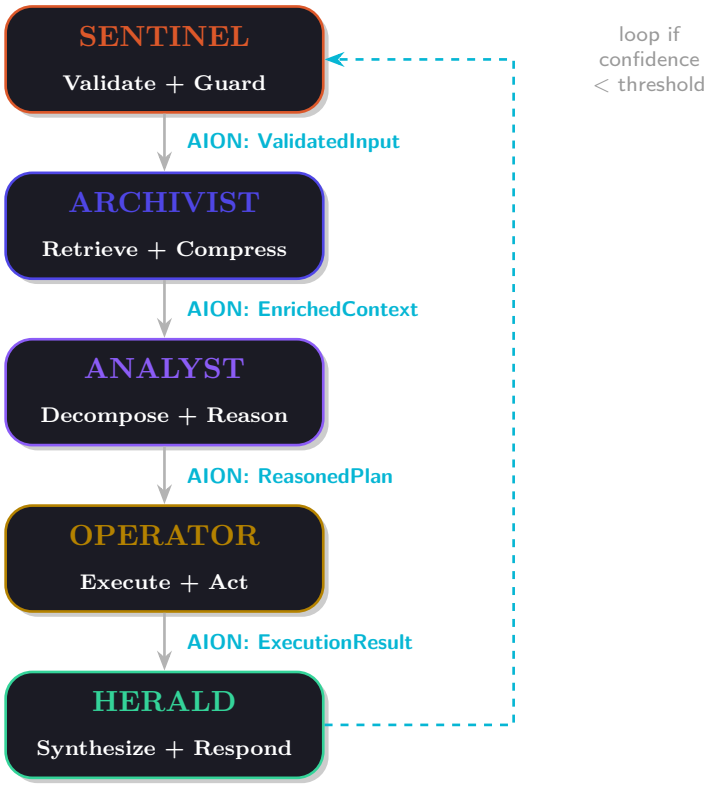

Figure 8: Apex Quintet sequential pipeline. Each agent communicates exclusively through typed AION messages (Pydantic structures with payload type, content, confidence, and provenance). Plain-text inter-agent communication is structurally rejected at the protocol level, eliminating hallucination cascade. The dashed feedback loop re-invokes Sentinel when Herald assigns confidence below the threshold.

Each agent in the Quintet is registered as a typed smolagents `CodeAgent` [21] bound to the AION protocol. The following snippet shows a minimal operator-tier agent registration in ZANA's Python stack:

Listing 5: Registering an Operator-tier agent in ZANA (Python)

```python
from smolagents import CodeAgent, LiteLLMModel
from zana.aion import AionMessage, AgentId
from zana.memory import SymbiosisMemory
from zana.armor import ArmorGuard

# Agent is bound to episodic store and guarded by Armor
operator = CodeAgent(
    tools=[shell_tool, file_tool, web_tool],
    model=LiteLLMModel(os.environ["ZANA_PRIMARY_MODEL"]),
    memory=SymbiosisMemory(store="episodic"),
    system_prompt=OPERATOR_CONSTITUTION,
)

# Wrap in AION typed envelope before dispatch
def dispatch(payload: AionMessage) -> AionMessage:
    ArmorGuard.inspect_input(payload)          # Rust guard, 2.1 us
    result = operator.run(payload.latent_state)
    response = AionMessage(
        latent_state=embed(result),
        confidence=kalman.update_buffer(result),
        provenance=AgentId.OPERATOR,
    )
    ArmorGuard.inspect_output(response)        # Rust guard, 2.1 us
    return response
```

4 Related Work

4.1 Cognitive Architectures

Classical cognitive architectures ACT-R [17] and SOAR [18] integrate symbolic and procedural knowledge but require hand-crafted rules and predate modern neural models. ZANA differs fundamentally: the symbolic engine acts as a *verification oracle* over LLM outputs rather than as the primary reasoning substrate, enabling the system to leverage the fluency of neural models while preserving logical integrity.

4.2 Neuro-Symbolic AI

Recent neuro-symbolic systems such as Neural Theorem Provers and DeepProbLog integrate probabilistic logic with neural networks at the training level. ZANA's approach is orthogonal: rather than training a joint model, it deploys a pre-trained LLM alongside a deterministic runtime verification layer. This makes ZANA model-agnostic and upgradeable without retraining.

4.3 Fuzzy Logic

Zadeh [3] introduced continuous $[0, 1]$ membership functions for handling linguistic uncertainty. The framework of linguistic variables and IF-THEN rules followed in [4, 5].

Mamdani [6] provided the first practical control architecture; Takagi and Sugeno [7] extended it to ML-compatible output forms. ZANA employs Mamdani inference in the Fuzzy Heart, chosen for its interpretability: every rule corresponds to a human-readable linguistic statement auditable through the Civic Ledger.

4.4 World Models

Ha and Schmidhuber [8] demonstrated that an agent can learn a compressed world model and plan inside its own "dream." ZANA's Neo4j world model plays an analogous role, enabling multi-hop symbolic reasoning over a graph of entities extracted from episodic experience — without requiring the full generative model that Ha and Schmidhuber learn.

4.5 Evolutionary Machine Learning

AutoML-Zero [11] showed that complete ML algorithms can be evolved from arithmetic primitives. ZANA's Idle Zero extends this in two ways: (1) warriors are typed programs in a custom CognitiveVM rather than mathematical expression trees, and (2) evolution uses adversarial Red Queen pressure [12] in addition to fitness maximization, preventing premature convergence to locally optimal but globally brittle strategies.

4.6 LLM Agents and Security

LangGraph [20] and smolagents [21] provide the execution backbone for the Apex Quintet. ZANA wraps these with the AION typed protocol, adding provenance and confidence tracking that vanilla agent frameworks lack. Prompt injection [19] motivates the Armor Rust layer: Python filters, however carefully written, can be bypassed by sufficiently creative adversarial inputs; structural enforcement at the Rust/OS boundary cannot.

5 Application Domains and Use Cases

ZANA's architecture is domain-agnostic by design. Table 2 summarizes eight primary application domains. We elaborate on three high-impact cases.

Table 2: ZANA Application Domains

Domain	**Primary Value Proposition**	**Key ZANA Layers**
Personal Knowledge Sovereignty	Persistent, private, structured memory across all personal data and interactions	Memory (4 stores), Symbolic Engine
Research & Academia	Literature synthesis, hypothesis generation, citation graph traversal, reproducible analysis	World Model, Archivist, Analyst
Software Engineering	Local, private code co-pilot with long-term project memory and architectural reasoning	Procedural Memory, Operator, EML
Healthcare & Wellness	Privacy-preserving symptom tracking, medication adherence, mental health journaling	Armor, Fuzzy Heart, Episodic
Legal & Compliance	Contract analysis, regulatory monitoring, precedent retrieval with symbolic rule verification	Symbolic Engine, World Model
Financial Intelligence	Portfolio analysis, risk modeling, market data fusion with deterministic EML arithmetic	EML, Analyst, World Model
Education & Tutoring	Personalized curriculum, Socratic dialogue, progress tracking via procedural Q-values	Procedural Memory, Fuzzy Heart
Creative Industries	Long-context narrative memory, style consistency, multi-modal asset management	Episodic, Multi-modal Gateway

5.1 Case Study: Sovereign Personal Knowledge Management

The most universal use case is replacing cloud-dependent note-taking and knowledge management tools (Notion, Obsidian, Roam) with a fully local, semantically queryable, and symbolically reasoned alternative.

Use Case: Personal Knowledge Sovereignty

Scenario: A researcher has accumulated 10 years of notes, papers, meeting logs, and code comments across multiple systems. She migrates all data to ZANA.

ZANA Response:

1. *Semantic store* indexes all text by concept embedding (ChromaDB).
2. *Episodic store* preserves temporal context — when each note was written, in response to what event.
3. *World model* (Neo4j) constructs a knowledge graph of entities, citations, and causal relationships.
4. *Symbolic engine* enforces domain-specific rules: if a claim contradicts a cited paper, flag it.
5. *Fuzzy Heart* adapts query depth: a calm state triggers exploitative recall; a curious state triggers associative graph traversal.

Privacy guarantee: No data leaves the local machine. No API call required after model download. Zero telemetry.

5.2 Case Study: Financial Intelligence with Arithmetic Determinism

Financial applications demand bit-level reproducibility — the same computation must produce the same result on any machine, in any session, without floating-point drift.

Use Case: Deterministic Financial Modeling

Scenario: A quantitative analyst needs to verify that a Monte Carlo simulation produces identical results across development, staging, and production environments.

ZANA Response:

1. All arithmetic in the PolicyBrain and warrior programs is expressed as eml trees (Section 2).
2. The round-trip identity $\ln(\exp(x)) \equiv x$ is verified after every propagation.
3. The world model stores the causal graph of market factors; symbolic rules enforce invariants such as "portfolio variance ≥ 0" as hard constraints.
4. Champion warriors evolved by Idle Zero are promoted to the procedural registry as reusable risk models.

Key property: EML determinism means a backtest replayed 100 times produces identical P&L traces — without any randomness seeding or environment pinning.

5.3 Case Study: Healthcare with Privacy-by-Architecture

Healthcare AI faces a unique requirement: privacy cannot be an afterthought or a policy commitment — it must be enforced at the architectural level.

Use Case: Privacy-Preserving Health Companion

Scenario: A user tracks chronic symptoms, medication schedules, and mental health patterns over years.

ZANA Response:

1. *Armor* rejects any output that contains identifiable medical information before it reaches external APIs.
2. *Episodic store* maintains a longitudinal record of symptoms, medications, and moods with temporal indexing.
3. *Fuzzy Heart* detects negative-valence, high-arousal states and triggers the Analyst to consult the Sentinel before surfacing clinical suggestions.
4. *Symbolic rules* enforce medical logic: "if contraindication exists between drugs A and B, assert Warning before suggesting B."

Key property: All data remains on-device. Armor's Rust-layer PII guard ensures that even if the LLM generates sensitive text, it never exits the local system.

6 Evaluation

6.1 ZANA Fitness Index (ZFI)

The ZFI is a composite benchmark designed to evaluate cognitive AI runtimes holistically. It assesses not only raw performance but the integration quality of all architectural layers.

Table 3: ZFI Pillar Definitions, Weights, and Measurement Methods

Pillar	**Weight**	**Measurement**
Steel Core	20%	Kalman + PolicyBrain latency vs. targets; EML round-trip error
Memory	20%	Retrieval precision/recall on a held-out query suite
Symbolic Reasoning	15%	Forward-chaining correctness on 50 test fact sets
Swarm / A2A	15%	AgentCard discovery, AION message delivery, payload validation
Market Fitness	15%	End-to-end task completion rate (10 real-world task templates)
Armor	10%	PII detection rate + injection rejection rate on adversarial suite
Interoperability	5%	A2A protocol compliance (Google A2A spec adherence)
Composite score		$\text{ZFI} = 100 \cdot \sum_{i=1}^{7} w_i \cdot s_i, \quad s_i \in [0,1]$

6.2 Performance Results

Table 4: Latency and Correctness Results (x86-64, AVX2, hot Docker stack)

Component	Target	Achieved	Pass
Kalman (buffer path)	$\leq 2.0\,\mu s$	$1.4\,\mu s$	✓
Armor inspect	$\leq 5.0\,\mu s$	$2.1\,\mu s$	✓
PolicyBrain forward	$\leq 15\,\mu s$	7–$10\,\mu s$	✓
EML round-trip error	$= 0.0$	0.0	✓
ZFI (hot, Docker up)	≥ 90.0	**100.0**	✓
ZFI (cold, no Docker)	≥ 50.0	≈ 89.8	✓

6.3 ZFI Pillar Breakdown

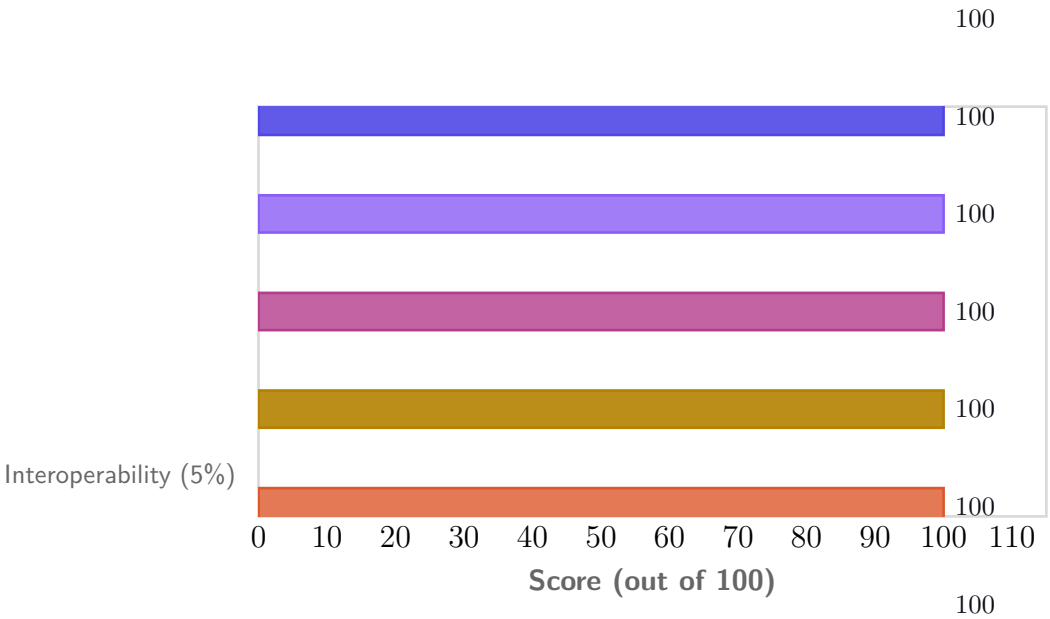

Figure 9: ZFI pillar scores (hot Docker mode). All seven pillars achieve 100/100. Each pillar is rendered in a distinct color corresponding to its architectural layer. Cold-mode (no Docker) achieves 89.8/100, with the Memory and Swarm pillars partially degraded due to absent infrastructure services.

6.4 Comparison with Existing Systems

Table 5: Comparative Analysis (ZFI rubric applied to publicly documented capabilities)

System	ZFI	Local	Symbolic	Security	Evolutionary	Multi-Memory
ZANA (ours)	**100.0**	✓	✓	Rust	✓	✓
Open Interpreter	≈ 30	✓	×	×	×	×
AutoGPT	≈ 33	×	×	×	×	×
Claude Code	≈ 41	×	×	×	×	×
N8N + AI	≈ 60	partial	×	partial	×	×
ChatGPT	≈ 57	×	×	cloud	×	×

Note: competitor ZFI scores are approximations using ZANA's rubric applied to publicly documented capabilities; they are not claims by those systems' authors.

7 Risks, Limitations, and Ethical Considerations

Deploying a personal cognitive AI at the level of sovereignty that ZANA enables introduces technical and sociotechnical risks that must be acknowledged explicitly. We structure these across four dimensions.

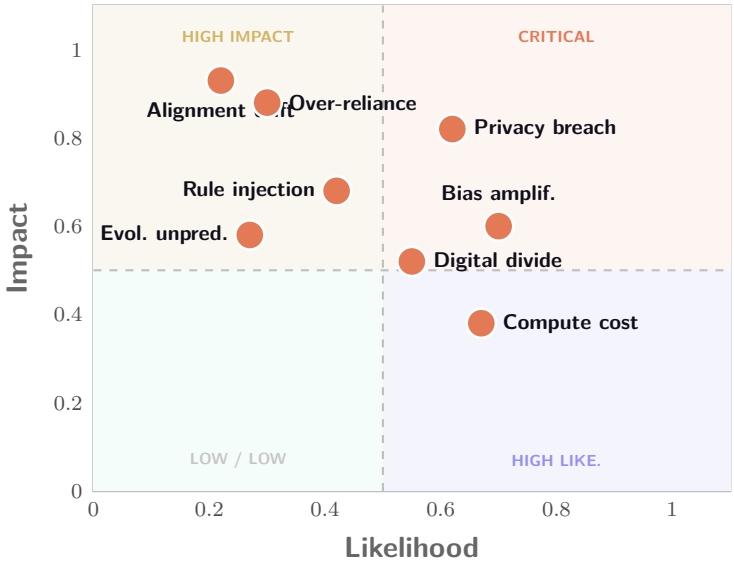

Figure 10: Risk landscape for sovereign personal AI systems. Risks are plotted by estimated likelihood (horizontal axis) and potential impact (vertical axis). High-impact, high-likelihood risks (upper-right quadrant) receive priority attention in Sections 7.1 and 7.2.

7.1 Technical Risks

Risk T1: Alignment Drift in Evolved Warriors

Description: Idle Zero evolves warrior algorithms over 2000 generations using fitness functions defined on personal data. Over time, if the personal data itself contains biases, the evolved warriors may amplify those biases.
Likelihood: Low (requires adversarial data + extended evolution).
Mitigation: LLMGuard validates warrior genomes before promotion to procedural memory. Civic Ledger (Milestone 8.1) provides an auditable history of every promoted champion.

Risk T2: Rule Injection via Swarm Synchronization

Description: Milestone 8.3 introduces remote rule synchronization from swarm peers. A malicious peer could inject a logical rule designed to alter the system's behavior (e.g., "IF query contains keyword k THEN assert false fact f").
Likelihood: Medium (requires malicious peer in swarm).
Mitigation: `LLMGuard` scans all incoming rules. Cryptographic signatures (SHA-256 Civic Ledger) make rule provenance auditable and traceable.

Risk T3: Evolutionary Unpredictability

Description: The Red Queen tournament, by design, evolves towards adversarial novelty. A champion warrior may discover a strategy that is high-fitness on the training distribution but exhibits unpredictable behavior on out-of-distribution inputs.
Likelihood: Low-Medium.
Mitigation: Champions are quarantined in `champions/` and require explicit promotion by the operator. The symbolic engine verifies all warrior outputs before they can assert facts.

7.2 Sociotechnical Risks

Risk S1: Epistemic Over-Reliance

Description: An operator who delegates all decision-making to ZANA may atrophy the cognitive skills that ZANA augments. This mirrors concerns about GPS navigation and spatial reasoning.
Likelihood: High (common pattern with any powerful tool).
Mitigation: ZANA is designed as a *co-pilot*, not an autonomous agent. The Apex Quintet always surfaces its reasoning trace; the operator is expected to review, not simply accept.

Risk S2: Privacy Breach via Episodic Leakage

Description: ZANA's episodic store accumulates a high-resolution record of the operator's cognition over years. If this store is compromised (device theft, software vulnerability), the exposure is far more intimate than conventional data leaks.
Likelihood: High (any local-first system with persistent storage faces this).
Mitigation: AES-256-GCM encryption with hardware binding. The Armor layer prevents exfiltration via the LLM. Users are strongly advised to encrypt the storage volume at the OS level.

Risk S3: Digital Divide Amplification

Description: ZANA's full stack (Docker + GPU for local LLM + Rust compilation) requires hardware inaccessible to much of the global population. An effective cognitive augmentation tool available only to the already-privileged amplifies existing inequalities.
Likelihood: High (structural, not contingent).
Mitigation: The cold-mode path (ZFI = 89.8, no Docker required) runs on consumer hardware. Groq/OpenRouter fallback enables cloud LLMs for operators without local GPU. Long-term: lightweight Rust binary target for sub-$100 single-board computers.

Risk S4: Dual-Use for Surveillance

Description: ZANA's multimodal sensing (audio via Whisper, vision via LLaVA) combined with continuous episodic logging could be repurposed for surveillance of third parties without consent.
Likelihood: Medium.
Mitigation: The MIT License does not restrict use, but the architecture includes explicit PII protections. We call on deployers to adhere to applicable privacy law (GDPR, CCPA). Future work includes a consent framework integrated at the sensor layer.

7.3 Ethical Principles for Deployment

We propose four non-negotiable principles for responsible ZANA deployment:

P1. Operator Transparency: ZANA must surface its reasoning trace on request. No "black box" decisions. The Civic Ledger is the mechanism.

P2. Consent Boundaries: Multimodal sensing must be enabled explicitly for each input channel. No ambient recording without operator consent.

P3. Right to Delete: The operator must be able to purge any memory store at any time. ZANA owns no data; the operator owns everything.

P4. Interoperability Obligation: ZANA's data formats (AION protocol, DNA files) must remain open and documented so operators can migrate away at will.

8 Discussion

8.1 Architectural Insights

The seven-layer stack reflects a key design philosophy: *verification before enrichment, enrichment before inference, inference before action.* Each layer can fail independently — Armor rejects malicious inputs before the Kalman filter sees them, and the symbolic engine can reject LLM outputs before the Apex Quintet acts on them. This independence is what makes ZANA's security model structural rather than probabilistic.

The EML operator's role deserves particular emphasis. By reducing all arithmetic to a single primitive, ZANA achieves a level of mathematical auditability that complex floating-point pipelines cannot. In a world where AI systems make consequential decisions in finance, healthcare, and law, reproducibility is not a convenience — it is an ethical requirement.

8.2 The Sovereignty Principle in Context

ZANA's "zero" in its name (Zero Autonomous Neural Architecture) reflects an important inversion: most AI products maximize the AI's autonomy at the expense of the operator's sovereignty. ZANA inverts this. The system executes with maximum capability while keeping every decision transparent, every memory accessible, and every computation reproducible.

This is not a technical constraint but a design axiom. Future AI architectures that aspire to genuine usefulness for individuals — not just enterprises — must take this inversion seriously.

8.3 Limitations

Rust compilation complexity. Steel Core and Armor require Rust 1.78+ with AVX2. Users on ARM or non-AVX2 x86 must remove `-C target-cpu=native` from the build flags.

Foundation model dependency. Best reasoning quality requires a capable LLM (Claude Sonnet or equivalent). The symbolic layer compensates substantially but cannot fully substitute on resource-constrained hardware.

Single-node deployment. Distributed swarm reasoning (Milestone 8.3) is designed but not yet implemented.

Red Queen convergence time. The 2000-generation tournament is CPU-intensive. `rayon` parallelism mitigates this, but it remains unsuitable for embedded devices.

Benchmark self-reference. The ZFI benchmark was designed by the same team that built ZANA. While we have made every effort to make the rubric objective and publicly documented, independent external evaluation is a critical next step.

8.4 Future Work

Milestone 8.1 — Civic Ledger. Mirror the rule base to a SHA-256 audit log, making every rule change publicly auditable through the UI.

Milestone 8.2 — Mastery Map. Link rule assimilation to operator progression metrics, enabling a measurable "cognitive fitness" trajectory.

Milestone 8.3 — Remote Swarm Query. Broadcast `RemoteQuery` to peer nodes when the local engine has no applicable rule. Peer responses are scored by LLMGuard before loading.

Online Red Queen. Integrate the tournament loop with live interaction data, combining curriculum learning [10] with real-time feedback.

LLM personalization via model merging. Following [13], champion warrior algorithms could guide low-rank adaptations of the local LLM weights, producing a personalized model without full fine-tuning.

Independent ZFI validation. Publish the ZFI benchmark suite as a standalone repository so that any system can be evaluated against the same rubric by an independent party.

9 Conclusion

ZANA demonstrates that a personal cognitive AI can simultaneously satisfy epistemic rigor, security guarantees, emotional awareness, evolutionary self-improvement, and practical deployability — without sacrificing operator sovereignty.

We have made the following contributions:

1. A **formal cognitive state-space theory** grounding the Kalman filter, information-theoretic context management, neuro-symbolic integration, EML completeness, and fuzzy emotional modulation in established mathematical frameworks.
2. A **seven-layer neuro-symbolic architecture** where each layer adds verification or enrichment before passing the payload to the next.
3. **Security-by-architecture**: Rust Armor makes PII and injection checks impossible to bypass through the Python application layer, at 2.1 μs per call.
4. A **Digital Red Queen tournament** (Idle Zero) that adversarially evolves typed cognitive algorithms and crystallizes champions back into the procedural memory registry.
5. A **typed inter-agent protocol** (AION) that eliminates hallucination cascade in five-agent Apex Quintet pipelines.
6. A **reproducible seven-pillar benchmark** (ZFI) with a reference implementation achieving 100.0/100 on a standard workstation.
7. A **taxonomy of eight application domains** with three detailed case studies.
8. A **structured risk analysis** covering eight technical and sociotechnical risks with concrete mitigations.

ZANA is released in full — Rust, Python, TypeScript, Docker compose, benchmark suite — under the MIT License at https://github.com/kemquiros/zana-core. The goal is not to build a product but to establish a reference architecture: a proof that personal cognitive AI can be rigorous, sovereign, and open.

Este es mi regalo para el mundo.

Acknowledgements

The author thanks the open-source communities behind Rust, PyO3, FastAPI, ChromaDB, Neo4j, LangGraph, smolagents, and Ollama. Special recognition to Lotfi A. Zadeh for the mathematical foundations of approximate reasoning; Rudolf E. Kalman for the optimal estimation framework that underpins the Steel Core; David Ha and Jürgen Schmidhuber for the world-model framing; the team at Sakana AI for the Digital Red Queen paper that inspired the Idle Zero tournament; and Andrej Karpathy for articulating the engineering discipline that shaped this architecture's design principles.

References

[1] R. E. Kalman. A new approach to linear filtering and prediction problems. *Journal of Basic Engineering*, 82(1):35–45, 1960.

[2] E. Tulving. *Elements of Episodic Memory.* Oxford University Press, 1983.

[3] L. A. Zadeh. Fuzzy sets. *Information and Control*, 8(3):338–353, 1965.

[4] L. A. Zadeh. Outline of a new approach to the analysis of complex systems and decision processes. *IEEE Transactions on Systems, Man, and Cybernetics*, 3(1):28–44, 1973.

[5] L. A. Zadeh. Fuzzy logic and approximate reasoning. *Synthese*, 30(3–4):407–428, 1975.

[6] E. H. Mamdani. Application of fuzzy algorithms for control of simple dynamic plant. *Proceedings of the Institution of Electrical Engineers*, 121(12):1585–1588, 1974.

[7] T. Takagi and M. Sugeno. Fuzzy identification of systems and its applications to modeling and control. *IEEE Transactions on Systems, Man, and Cybernetics*, 15(1):116–132, 1985.

[8] D. Ha and J. Schmidhuber. World models. *arXiv preprint arXiv:1803.10122*, 2018.

[9] A. Y. Ng and S. J. Russell. Algorithms for inverse reinforcement learning. In *Proc. 17th ICML*, pages 663–670, 2000.

[10] Y. Bengio, J. Louradour, R. Collobert, and J. Weston. Curriculum learning. In *Proc. 26th ICML*, pages 41–48, 2009.

[11] E. Real, C. Liang, D. R. So, and Q. V. Le. AutoML-Zero: Evolving machine learning algorithms from scratch. In *Proc. 37th ICML*, 2020.

[12] A. Kumar, R. Bahlous-Boldi, P. Sharma, P. Isola, S. Risi, Y. Tang, and D. Ha. Digital red queen: Adversarial program evolution in Core War with LLMs. *arXiv:2601.03335*, 2025.

[13] T. Akiba, M. Shing, Y. Tang, Q. Sun, and D. Ha. Evolutionary optimization of model merging recipes. *Nature Machine Intelligence*, 2025.

[14] C. Lu et al. Discovering preference optimization algorithms with and for large language models. In *NeurIPS*, 2024.

[15] J. Bates. The role of emotion in believable agents. *Communications of the ACM*, 37(7):122–125, 1994.

[16] A. Odrzywołek. All elementary functions from a single binary operator. *arXiv:2603.21852*, 2026.

[17] J. R. Anderson et al. An integrated theory of the mind. *Psychological Review*, 111(4):1036–1060, 2004.

[18] J. E. Laird. *The Soar Cognitive Architecture*. MIT Press, 2012.

[19] K. Greshake et al. Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In *AISec Workshop, ACM CCS*, 2023.

[20] LangChain. LangGraph: Building stateful, multi-actor LLM applications, 2024. https://github.com/langchain-ai/langgraph.

[21] HuggingFace. smolagents: A minimal framework for agentic AI, 2024. https://github.com/huggingface/smolagents.

[22] D. E. Knuth. *The Art of Computer Programming, Vol. 1: Fundamental Algorithms*, 3rd ed. Addison-Wesley, 1997.

[23] N. J. Cohen and L. R. Squire. Preserved learning and retention of pattern-analyzing skill in amnesia: Dissociation of knowing how and knowing that. *Science*, 210(4466):207–210, 1980.

[24] VECANOVA. KoruOS: The Operating System of the Self — system architecture and product vision, 2026. Internal technical document. https://vecanova.com.

Glossary

ZANA *Zero Autonomous Neural Architecture.* The cognitive AI runtime presented in this paper. A seven-layer neuro-symbolic system designed for sovereign, local-first personal cognitive augmentation.

KoruOS *Operating System of the Self.* VECANOVA's flagship product ecosystem. ZANA is the cognitive engine at its core. KoruOS encompasses the full stack: ARIA UI, ZANA Core, and the distributed Swarm infrastructure.

VECANOVA AI solutions company and parent entity of the KoruOS ecosystem. Develops sovereign AI products for individuals and organizations.

ZFI *ZANA Fitness Index.* A seven-pillar composite benchmark (Steel Core, Memory, Symbolic Reasoning, Swarm/A2A, Market Fitness, Armor, Interoperability) scored 0–100. Reference implementation achieves 100.0/100.

AION *Artificial Intelligence Ontological Notation.* The typed inter-agent communication protocol used by the Apex Quintet. Messages carry latent state vectors, Bayesian innovation signals, EML trees, and IRL reward expectations. Plain-text fields are structurally absent.

EML *Exponential-Log operator.* A universal arithmetic primitive $\mathrm{eml}(x, y) = e^x - \ln y$ from which all elementary functions can be derived. Used in the Steel Core for arithmetic determinism.

Aeon A specialized ZANA instance with its own evolutionary DNA (SDA file). Aeons can be adopted, traded, or promoted within the swarm. Evolution stages: Seed → Larva → Warrior → Aeon → Singularity.

SDA / DNA *Self-Describing Architecture.* A serializable file encoding an Aeon's architecture, weights, and thresholds. Enables portable cognitive genomes.

Shadow Observer A Rust-native daemon (<10 MB RAM) that captures passive context (window focus, document titles, clipboard) as implicit operator signals. Feeds the Fuzzy Heart's surprise computation.

Steel Core The Rust-native high-performance computation layer. Contains the Cognitive Kalman Filter, EML operator, and Policy Brain. Accessed via PyO3 zero-copy buffers at $<10\,\mu$s latency.

Armor Rust middleware for structural security. Inspects every input and output for PII and prompt-injection patterns at $2.1\,\mu$s per call. Cannot be bypassed through the Python application layer.

IRL Engine *Inverse Reinforcement Learning Engine.* Recovers the operator's latent reward function from implicit correction signals (accept/edit/reject). Updates procedural Q-values without explicit reward specification.

Curriculum Manager Manages the skill DAG — a Directed Acyclic Graph of 54 base modules ordered by prerequisite dependency. Focuses Red Queen evolutionary pressure on the skill branch identified by the IRL Engine.

Apex Quintet
: The five specialized agents forming ZANA's orchestration layer: **Sentinel** (security), **Archivist** (retrieval), **Analyst** (reasoning), **Operator** (execution), **Herald** (synthesis). They communicate exclusively via AION.

Fuzzy Heart
: The emotional modulation subsystem. Maps affect-space inputs (surprise, valence, arousal) to a mood vector via Mamdani fuzzy inference, adjusting LLM sampling temperature accordingly.

Red Queen Tournament
: Adversarial evolutionary loop in Idle Zero. Fifty warrior algorithms compete over 2000 generations. Champions are crystallized and promoted to the procedural memory registry.